Forecasting Daily Stock Closing Prices with a Stacked LSTM Network

An empirical study on Tesla, Inc. (December 2017 – December 2022)

Ayush Kumar Upadhyay

Independent project report · Experiments conducted December 2022 · Report written October 2026

Code and data: https://github.com/ayushupadhyayk/StockPriceForecasting

---

Abstract

This report studies whether a stacked Long Short-Term Memory (LSTM) network can learn to forecast the daily closing price of a volatile stock from its own price history. I collected five years of daily price data for three stocks, Tesla (TSLA), Apple (AAPL) and Alphabet (GOOGL), covering 1,259 trading days each from 4 December 2017 to 2 December 2022. Tesla was used for modelling. A three-layer LSTM with 50,851 parameters was trained to predict the next day's closing price from the previous 100 days, using a chronological split of 65% training data (4 December 2017 – 5 March 2021) and 35% test data (8 March 2021 – 2 December 2022). Over 100 training epochs, the mean squared error on scaled prices fell from 0.0197 to 0.0011, and the test error reached a root mean squared error (RMSE) of about **$72–75**, roughly 10% of the average test-period price of $745. The model followed the overall movement of the price closely on unseen data, but a recursive 30-day forecast predicted a rise from $194.86 to $230.93 while the real price fell. The data analysis also showed that the unadjusted prices contain two Tesla stock splits that look like one-day crashes of 77.5% and 66.8%. These results and limitations shaped my 7th semester Major Project-1, a stock trend prediction web app.

Keywords: time series forecasting, LSTM, recurrent neural networks, stock prices, deep learning

---

1. Introduction

Stock prices are noisy, non-stationary time series: their average level and volatility change over time, and they react to news that isn't visible in past prices. Forecasting them is a standard benchmark for sequence models. Recurrent neural networks, and LSTMs in particular, can learn patterns across many past time steps, which makes them a natural choice for price data.

The aim of this project was to build a complete forecasting pipeline myself, from collecting the data to evaluating the model, and to understand honestly what such a model can and cannot do. I set out to answer three research questions:

1. **Learnability.** Can a stacked LSTM trained only on past closing prices follow the price of a highly volatile stock on data it has never seen?
2. **Forecasting.** How does the model behave when used to forecast several weeks ahead?
3. **Pitfalls.** What practical issues in the data and the evaluation affect results like these?

2. Project overview

2.1 What I did

In summary, I:

1. **Collected** five years of daily price data for Tesla, Apple and Alphabet from the Tiingo financial data API and saved each to a CSV file.
2. **Explored and prepared** the Tesla closing prices: plotted them, scaled them to the range 0–1, and split them chronologically into training and test periods.
3. **Turned the series into a supervised learning problem** by building sliding windows of 100 days, each labelled with the following day's price.
4. **Designed and trained** a three-layer stacked LSTM network in TensorFlow/Keras for 100 epochs.
5. **Evaluated** the model on the unseen test period by converting its predictions back to dollars and comparing them with the actual prices.
6. **Forecast** the next 30 trading days by feeding the model's own predictions back into it.
7. **Analysed** the results and the data, including stock splits, error levels and limitations.

2.2 How I did it: the workflow

Stage	What happens	Input	Output	Tools
1. Data collection	Download daily prices through the Tiingo API	Ticker symbol, API key	`TSLA.csv`, `APPL.csv`, `GOOGL.csv`	pandas-datareader
2. Loading	Read the CSV and keep the `close` column	`TSLA.csv`	1,259 closing prices	pandas
3. Visualisation	Plot the full price history	Closing prices	Price chart	Matplotlib
4. Scaling	Map prices to the range 0–1	Closing prices	Scaled series	scikit-learn `MinMaxScaler`
5. Splitting	First 65% for training, last 35% for testing	Scaled series	818 + 441 days	NumPy
6. Windowing	Build 100-day input windows and next-day targets	Each split	717 training and 340 test samples	NumPy
7. Model building	Stack three LSTM layers and a dense output layer	Window shape (100, 1)	Compiled model, 50,851 parameters	TensorFlow / Keras
8. Training	Fit for 100 epochs, batch size 64, checking test loss each epoch	Training windows	Trained model, loss history	Keras
9. Evaluation	Predict both periods, convert to dollars, plot against actual prices	Trained model	Figure 1, error figures	Keras, Matplotlib
10. Forecasting	Predict 30 days ahead, one day at a time	Last 100 days of data	30-day forecast, Figure 2	Keras, NumPy

The full code, with every output, is in the notebook `Stack LSTM.ipynb` (see Appendix B for a map from these stages to notebook cells).

3. Background: how an LSTM works

3.1 Why ordinary networks struggle with sequences

A standard feed-forward neural network sees each input independently and has no notion of order. A **recurrent neural network (RNN)** processes a sequence one step at a time and carries a *hidden state* from each step to the next, so earlier values can influence later outputs. In practice, plain RNNs struggle to learn long-range patterns: during training, the gradients that carry the error signal back through many time steps shrink towards zero or blow up (the *vanishing* and *exploding gradient* problems). An RNN looking at 100 days of prices therefore mostly "forgets" the early days.

3.2 The LSTM cell

The **Long Short-Term Memory** network, introduced by Hochreiter and Schmidhuber [1], solves this with a separate **cell state**, a memory line that runs through the whole sequence and is changed only through carefully controlled updates. At each time step t, the cell receives the current input x_t (here, one day's scaled price) and the previous hidden state h_{t-1}, and uses three **gates** to decide what to remember. Each gate is a small layer with a sigmoid activation σ, which outputs values between 0 (block everything) and 1 (let everything through).

Component	Equation	What it does
Forget gate	$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	Decides how much of the old memory to keep
Input gate	$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	Decides how much new information to store
Candidate memory	$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$	Proposes new information to add
Cell state update	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	Keeps part of the old memory and adds part of the new
Output gate	$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	Decides what part of the memory to output
Hidden state	$h_t = o_t \odot \tanh(c_t)$	The cell's output for this step

Here W and b are learned weights and biases, $[\cdot,\cdot]$ joins two vectors, and $\odot$ is element-wise multiplication. Because the cell state is updated by addition rather than repeated multiplication, the error signal can flow back across many time steps without vanishing, which lets an LSTM learn patterns spanning the full 100-day window.

An intuition for stock prices. As the LSTM reads 100 days of prices, the forget gate can learn to drop the influence of an old spike once the price has settled, the input gate can learn to store a new upward run, and the output gate decides how much of that stored trend should influence the prediction for the next day.

3.3 Stacking LSTM layers

In a **stacked LSTM**, each LSTM layer passes its full sequence of hidden states (one per day) to the next layer, instead of only its final state. Lower layers can learn short-term patterns in the raw prices, while higher layers can combine them into longer-term trends. Only the last LSTM layer outputs a single summary vector, which a dense layer turns into the final prediction. Each LSTM layer with n units and m inputs has $4 \times (n \times (m + n) + n)$ parameters: four sets of weights and biases, one for each gate and the candidate memory.

3.4 LSTMs for financial data

LSTMs have been widely applied to financial time series. Fischer and Krauss [2], for example, found that LSTM networks outperformed memory-free methods such as random forests and logistic regression at predicting the direction of S&P 500 stocks. That kind of evidence motivated choosing an LSTM for this project.

4. Datasets

4.1 Source and collection

All data came from the **Tiingo** financial data API, downloaded with the `get_data_tiingo` function of the pandas-datareader library in December 2022 and saved as CSV files. Each file holds the full daily history of one stock over the same five-year window.

Dataset	File	Company	Exchange ticker	Used for
1	`TSLA.csv`	Tesla, Inc.	TSLA (NASDAQ)	**Model training, testing and forecasting**
2	`APPL.csv`	Apple Inc.	AAPL (NASDAQ)	Collected for comparison and data analysis (the filename has a typo for AAPL)
3	`GOOGL.csv`	Alphabet Inc. (Class A)	GOOGL (NASDAQ)	Collected for comparison and data analysis

4.2 Data range and size

Dataset	First trading day	Last trading day	Trading days (rows)	Calendar span
Tesla	4 Dec 2017	2 Dec 2022	1,259	1,824 days (about 5 years)
Apple	4 Dec 2017	2 Dec 2022	1,259	1,824 days
Alphabet	4 Dec 2017	2 Dec 2022	1,259	1,824 days

All three files contain exactly the same trading days. There are no missing trading days: the longest gap between consecutive rows is four calendar days, which is a long weekend.

4.3 Data dictionary

Every file has the same 14 columns:

Column	Meaning
`symbol`	Stock ticker
`date`	Trading date (timestamp, UTC)
`open`, `high`, `low`, `close`	The day's opening, highest, lowest and closing price in US dollars, as quoted on that day
`volume`	Number of shares traded that day
`adjOpen`, `adjHigh`, `adjLow`, `adjClose`	The same prices adjusted for later stock splits and dividends, so the whole history is on one comparable scale
`adjVolume`	Volume adjusted for stock splits
`divCash`	Cash dividend paid per share on that day (0 if none)
`splitFactor`	Stock split ratio taking effect that day (1.0 if none)

The model uses only the **unadjusted `close`** column.

4.4 Closing price statistics (unadjusted `close`)

Dataset	Minimum (date)	Maximum (date)	Mean	Median	Standard deviation	First	Last
Tesla	$167.87 (21 Nov 2022)	$2,238.75 (27 Aug 2020)	$561.40	$427.64	$331.29	$305.20	$194.86
Apple	$106.84 (18 Sep 2020)	$506.09 (26 Aug 2020)	$191.22	$172.43	$68.74	$169.80	$147.81
Alphabet	$83.43 (3 Nov 2022)	$2,996.77 (18 Nov 2021)	$1,548.48	$1,305.64	$738.20	$1,011.87	$100.44

4.5 Split-adjusted price statistics (`adjClose`)

Dataset	Minimum (date)	Maximum (date)	Mean	First	Last	Change over 5 years
Tesla	$11.93 (3 Jun 2019)	$409.97 (4 Nov 2021)	$129.87	$20.35	$194.86	+857.7%
Apple	$34.31 (3 Jan 2019)	$180.96 (3 Jan 2022)	$95.37	$40.36	$147.81	+266.2%
Alphabet	$49.23 (24 Dec 2018)	$149.84 (18 Nov 2021)	$85.13	$50.59	$100.44	+98.5%

Tesla was chosen for modelling because it was by far the most volatile of the three: its adjusted price rose almost tenfold over the period, with large swings in both directions.

4.6 Trading volume

Dataset	Minimum	Maximum	Mean daily volume
Tesla	2,465,629	222,126,194	22,958,699
Apple	11,362,045	332,607,163	61,870,310
Alphabet	465,638	88,279,042	4,012,718

4.7 Corporate actions: stock splits and dividends

Dataset	Event	Date	Detail
Tesla	Stock split	31 Aug 2020	5-for-1
Tesla	Stock split	25 Aug 2022	3-for-1
Apple	Stock split	31 Aug 2020	4-for-1
Apple	Dividends	Quarterly	20 cash dividends over the period
Alphabet	Stock split	18 Jul 2022	20-for-1

Stock splits matter a great deal for this study. A split multiplies the number of shares and divides the quoted price, without changing the company's value. In Tesla's **unadjusted** closing price, the two splits appear as one-day falls of **77.5%** (31 August 2020) and **66.8%** (25 August 2022). In the split-adjusted series, the largest one-day fall over the whole period is 21.1% (8 September 2020). A model trained on unadjusted prices therefore sees two crashes that never happened. This is analysed further in Sections 9 and 10.

5. Experimental setup

5.1 Date ranges used in the experiment

Part of the experiment	From	To	Size
Full Tesla dataset	4 Dec 2017	2 Dec 2022	1,259 trading days
Training period (first 65%)	4 Dec 2017	5 Mar 2021	818 trading days
Test period (last 35%)	8 Mar 2021	2 Dec 2022	441 trading days
Training targets (days the model learned to predict)	30 Apr 2018	4 Mar 2021	717 samples
Test targets (days the model was evaluated on)	29 Jul 2021	1 Dec 2022	340 samples
Forecast seed (input for the forecast)	14 Jul 2022	2 Dec 2022	Last 100 trading days
Forecast horizon	5 Dec 2022	about 18 Jan 2023	30 trading days

Training targets start in April 2018 because the first 100 days of each period are needed to form the first input window. The 2020 Tesla split falls inside the training period, and the 2022 split falls inside the test period.

5.2 Preprocessing

1. **Feature selection.** The unadjusted daily close was used as the only input feature.
2. **Scaling.** Prices were scaled to [0, 1] with min-max scaling [3], $x' = (x - \min) / (\max - \min)$, using the series minimum of $167.87 and maximum of $2,238.75. Neural networks train more reliably on small, similar-sized inputs.
3. **Chronological split.** The first 65% of days were used for training and the remaining 35% for testing, never shuffling, so the model is always evaluated on data from *after* the data it learned from, as it would be in real use.
4. **Windowing.** Each sample is a sliding window of 100 consecutive scaled prices, with the next day's scaled price as the target. For example, the first training sample uses days 1–100 to predict day 101,

the second uses days 2–101 to predict day 102, and so on.

5. **Reshaping.** The windows were reshaped to three dimensions, (samples, 100 time steps, 1 feature), the input format LSTM layers expect. This gave arrays of shape (717, 100, 1) for training and (340, 100, 1) for testing.

5.3 Model architecture

The model is a Keras `Sequential` stack [4] of three LSTM layers followed by one dense layer.

```
Input: 100 days × 1 feature (scaled closing price)
        │
   LSTM 1: 50 units, returns the full sequence      → output (100, 50)
        │
   LSTM 2: 50 units, returns the full sequence      → output (100, 50)
        │
   LSTM 3: 50 units, returns only the final state   → output (50)
        │
   Dense: 1 unit                                    → next day's scaled price
```

Layer	Configuration	Output shape	Parameters	Calculation
LSTM 1	50 units, returns sequences	(100, 50)	10,400	4 × (50 × (1 + 50) + 50)
LSTM 2	50 units, returns sequences	(100, 50)	20,200	4 × (50 × (50 + 50) + 50)
LSTM 3	50 units	(50)	20,200	4 × (50 × (50 + 50) + 50)
Dense	1 unit, linear	(1)	51	50 × 1 + 1
Total			**50,851**	All trainable

6. Training

6.1 Configuration

Setting	Value	Why
Loss function	Mean squared error (MSE)	Standard for predicting a continuous value; penalises large errors heavily
Optimiser	Adam	Adapts the learning rate for each weight; a reliable default for LSTMs
Epochs	100	Enough for the loss to level off
Batch size	64	12 weight updates per epoch (717 samples ÷ 64, rounded up)
Validation data	The 340 test windows	The test loss was tracked after every epoch
Framework	TensorFlow 2.11 with Keras, Python 3, Anaconda on Windows	

6.2 How training works

In each epoch, the 717 training windows were fed to the network in 12 batches. For each batch, the network predicted the next-day price for every window, the MSE between those predictions and the true prices was calculated, and the Adam optimiser adjusted all 50,851 weights slightly to reduce that error (backpropagation through time). After every epoch, the model was run on the 340 test windows to measure the validation loss, without learning from them. The first epoch took about 9 seconds and later epochs about 2 seconds each.

6.3 Training progress

Epoch	Training loss (MSE, scaled)	Validation loss (MSE, scaled)
1	0.0197	0.0142
2	0.0118	0.0043
3	0.0081	0.0039
78 (lowest training loss)	0.0010	0.0016
82 (lowest validation loss)	0.0010	0.0012
100 (final)	0.0011	0.0013

The loss dropped sharply in the first three epochs and then decreased slowly before levelling off. Training and validation loss stayed close together throughout, so the model was not badly overfitting: it performed nearly as well on unseen data as on the data it learned from.

7. How the model was used

7.1 Evaluation on known data

After training, the model predicted the next-day price for every training and test window. Because it outputs scaled values, the predictions were converted back to dollars with the inverse of the min-max scaling, $x = x' \times (\max - \min) + \min$, and plotted against the actual closing prices (Figure 1).

7.2 Recursive 30-day forecast

To look beyond the available data, the model was used **recursively**:

1. Take the last 100 days of scaled prices (14 July – 2 December 2022) as the input window.
2. Predict the next day's price.
3. Append the prediction to the end of the window and drop the oldest day.
4. Repeat steps 2–3 until 30 future days have been predicted.
5. Convert the 30 predictions back to dollars and plot them after the historical prices (Figure 2).

From the second step onwards, the model's inputs include its own earlier predictions rather than real prices.

8. Results

8.1 Error and deviation

The training log records the MSE on scaled prices. Taking the square root gives the **root mean squared error (RMSE)**, the typical size of the deviation between predicted and actual prices. Multiplying by the scaling range ($2,070.88) converts it to dollars.

Measure	Scaled MSE	Scaled RMSE	RMSE in dollars
Training error, final epoch	0.0011	0.0332	**$68.68**
Test error, epoch 1 (before learning)	0.0142	0.1192	$246.77
Test error, best epoch (82)	0.0012	0.0346	**$71.74**
Test error, final epoch (100)	0.0013	0.0361	**$74.67**

Training cut the test error by about 70%, from $246.77 after the first epoch to $74.67 at the end.

Putting the deviation in context. Over the 340 test days (29 July 2021 to 1 December 2022):

Test-period statistic	Value
Average closing price	$745.21
Standard deviation of the closing price	$288.54
Price range	$167.87 – $1,229.91
Average size of a day-to-day price move (excluding the split day)	$21.67

- The test RMSE of about **$72–75 is roughly 10% of the average price**.
- It is about **a quarter of the price's standard deviation** ($288.54), so the model explains most of the price's variation: it is far more accurate than simply guessing the average price.
- It is about **3.4 times a typical daily price move** ($21.67), so the model is not precise enough to predict individual day-to-day changes.
- The test period contains the 25 August 2022 split, when the quoted price fell by two-thirds overnight. A next-day model cannot anticipate this, so that single day likely accounts for a noticeable part of the test error.

8.2 Model fit

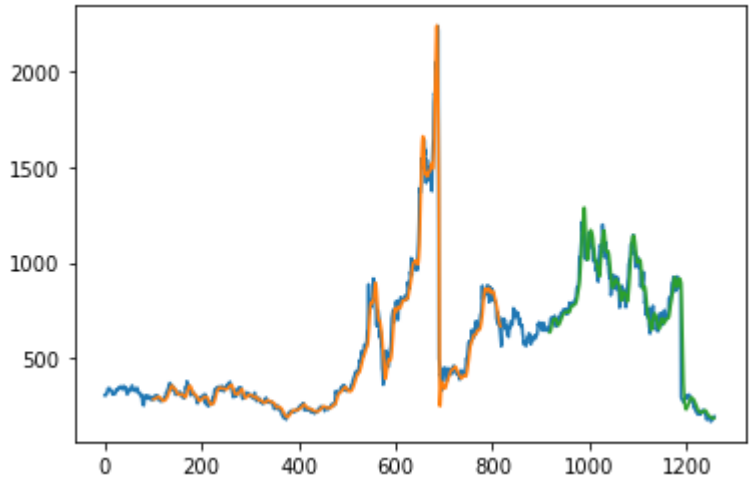


Figure 1. Actual Tesla closing price (blue), with model predictions on the training windows (orange) and the unseen test windows (green). The x-axis is trading days from 4 December 2017; the y-axis is the unadjusted closing price in dollars.

The predictions follow the actual price closely on both periods, including the surge of 2020, the peak of late 2021 and the decline through 2022. The main visible difference is a short lag: the model tends to react to a move a day or so after it happens. The sharp drops near day 680 and day 1,200 are the two stock splits.

8.3 Thirty-day forecast

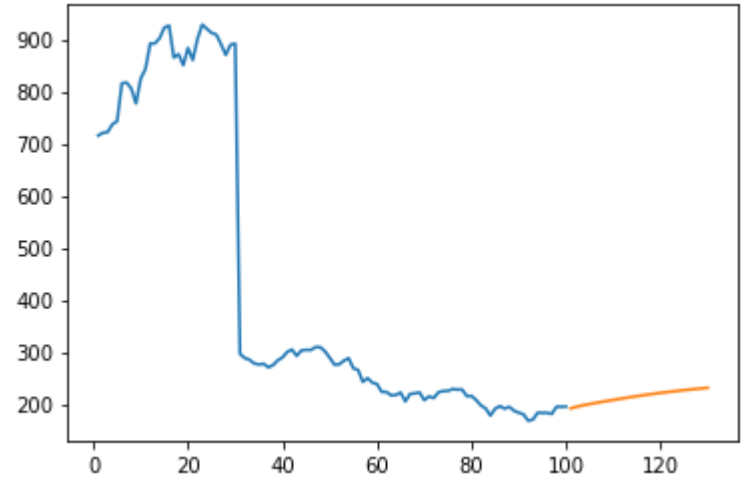


Figure 2. The last 100 trading days of actual prices (blue, 14 July – 2 December 2022) followed by the recursive 30-day forecast (orange, 5 December 2022 – about 18 January 2023). The steep fall in the blue line near day 30 is the 3-for-1 split of 25 August 2022.

Forecast detail	Value
Last actual closing price (2 Dec 2022)	$194.86
Forecast for day 1 (5 Dec 2022)	$191.42
Forecast for day 30 (about 18 Jan 2023)	$230.93
Forecast change over 30 days	+18.5%

The forecast is a smooth, steadily rising line with none of the day-to-day volatility of the real price. **In reality, Tesla's price fell sharply over this period**, closing 2022 at about $123 (public market data, not part of the dataset). The forecast therefore got the direction wrong, which shows how unreliable recursive multi-week forecasts from price history alone can be.

8.4 A note on the RMSE printed in the notebook

The notebook also prints RMSE values of 596.67 (training) and 802.92 (test). These compare predictions that were converted back to dollars with targets that were still on the 0–1 scale, so they don't measure the model's real error. The dollar figures in Section 8.1 replace them.

9. Discussion

Learnability (Question 1). Yes, within limits. On unseen data, the stacked LSTM followed the level and direction of Tesla's price closely, with test error close to training error and around 10% of the average price. However, much of this fit comes from the model leaning heavily on the most recent prices: a model that lags a day behind can look accurate on a chart while having little real predictive skill. Its error of roughly three times a typical daily move confirms that it cannot predict day-to-day changes. A simple baseline that predicts tomorrow's price as today's (the persistence forecast) would be needed to measure exactly how much the LSTM adds.

Forecasting (Question 2). Used recursively, the model produced a smooth trend that drifted upwards, while the real price fell. Each step feeds earlier predictions back into the input, so errors compound and the output regresses towards a smooth line. Multi-week forecasts from this approach are not reliable.

Pitfalls (Question 3). The most important finding came from the data rather than the model. Training on unadjusted prices put two artificial crashes into the series, one in the training period and one in the test period. This teaches the model false price dynamics and inflates the test error. Using the split-adjusted close is a simple and essential fix. Fitting the scaler on the full series and miscalculating the error metric were further evaluation pitfalls (Section 10).

10. Limitations

- **Unadjusted prices.** The stock splits in Section 4.7 distort both training and evaluation.
- **Scaler leakage.** The min-max scaler was fitted on the full series before the train/test split, so the test period's price range leaked into training. It should be fitted on training data only.
- **Error metric in the notebook.** The printed RMSE values compared data on different scales (Section 8.4). The error figures in this report are derived from the logged MSE instead.
- **Window mismatch.** The model was trained on 100-day windows, but the forecasting loop passed 99-day windows.
- **No baselines.** The model was not compared with persistence, moving-average or ARIMA forecasts.
- **Single split, single run.** Results come from one chronological split and one training run. Walk-forward validation and repeated runs with different random seeds would give more reliable estimates.
- **One stock, one feature.** Only Tesla's closing price was modelled. Volume, the other price fields, the Apple and Alphabet data, and outside information such as news or market indices were not used as

inputs.

11. Impact: how this study shaped my Major Project

The lessons from this study directly informed my 7th semester **Major Project-1**, StockPredictionWebApp, a Streamlit web app for stock trend prediction. It keeps the core pipeline developed here (100-day windows, min-max scaling, a chronological split and a stacked LSTM) and addresses several of the findings:

Finding in this study	Change in the Major Project
A small model with no regularisation	A deeper four-layer LSTM (50, 60, 80 and 120 units) with dropout after each layer
Five years of one stock	About 13 years of daily data, with any ticker selectable in the app
Recursive forecasts compound error and got the direction wrong	Day-by-day predictions evaluated against real prices
Retraining on every run	A trained model saved once and loaded by the app
Results only in a notebook	An interactive app with 100-day and 200-day moving-average trend analysis

Remaining future work includes split-adjusted prices, a training-only scaler, persistence and ARIMA baselines, walk-forward validation, and multivariate inputs such as volume and market indices.

12. Conclusion

This project built and evaluated a complete deep learning pipeline for stock price forecasting. Using five years of daily data for Tesla, Apple and Alphabet (4 December 2017 to 2 December 2022, 1,259 trading days each), I trained a three-layer stacked LSTM with 50,851 parameters to predict Tesla's next-day closing price from the previous 100 days.

The main findings are:

1. **The model learned the shape of the price series.** Training reduced the test error by about 70%, to an RMSE of about $72–75, roughly 10% of the average test-period price and a quarter of the price's natural variation. Predictions tracked the real price closely on data the model had never seen, with training and test error close together.
2. **It could not predict precise daily moves.** The error is several times larger than a typical daily price change, and the predictions lag the real price by about a day.
3. **Long-range forecasts were unreliable.** The recursive 30-day forecast rose 18.5% while the real price fell sharply, because errors compound when the model feeds on its own predictions.
4. **Data quality mattered as much as the model.** Unadjusted stock splits created two artificial crashes in the data, and scaler leakage and a mis-scaled error metric could each have given a misleading picture of performance.

Overall, LSTMs can model the general behaviour of a stock's price, but predicting exact future prices from price history alone is very limited, and careful data preparation and honest evaluation are essential. These

lessons carried directly into the design of my Major Project.

References

1. S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
2. T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018.
3. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
4. M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.

Appendix A: Reproducibility

Item	Location or value
Notebook with all code and outputs	`Stack LSTM.ipynb`
Data	`TSLA.csv`, `APPL.csv`, `GOOGL.csv` (Tiingo, 4 Dec 2017 – 2 Dec 2022)
Environment	Python 3, TensorFlow 2.11, scikit-learn, pandas, NumPy, Matplotlib, pandas-datareader

All figures, training values and forecast values in this report come from the outputs saved in the notebook. Dataset statistics, date ranges and the stock split analysis were computed from the CSV files. The dollar error figures were derived from the logged MSE values, as described in Section 8.1.

Appendix B: Notebook map

Stage (Section 2.2)	Notebook cells
Data collection	0–2
Loading and inspection	3–8
Visualisation	9
Scaling	10–13
Train/test split	14–17
Windowing and reshaping	18–22
Model building	23–27
Training	28
Evaluation and plotting	29–34
30-day forecast	35–46